TECHNICAL DOCUMENTATION

RARITONE BACKEND PROJECT

MEKA HRISHI TEJA CHOWDARY

1. INTRODUCTION

The fashion industry has experienced significant digital transformation over the past decade, with online shopping becoming one of the most preferred methods for purchasing clothing and accessories. Although e-commerce platforms offer convenience and accessibility, customers often face challenges such as selecting the correct clothing size, understanding product appearance, and making confident purchasing decisions without physically trying products.

To address these challenges, the Raritone platform was developed as an intelligent virtual fashion solution. The platform combines product management, user personalization, wardrobe organization, avatar support, body measurement management, and virtual try-on capabilities into a single ecosystem. By utilizing modern web technologies, the platform aims to provide users with a more engaging and personalized shopping experience.

The backend system plays a critical role in ensuring smooth communication between users, frontend applications, databases, and cloud services. It is responsible for processing business logic, managing product information, handling user requests, storing application data, and exposing secure APIs for various platform functionalities.

This documentation primarily focuses on my contributions to the backend development of the Raritone platform. My work was centered on the Product Module, Product APIs, Order Module support, and Virtual Try-On workflows. These components are essential for managing product catalogs, supporting purchasing processes, and enabling future AI-powered virtual fashion experiences.

The backend was implemented using Node.js, Express.js, and MongoDB, following a modular architecture that promotes scalability, maintainability, and clean code organization. Security measures such as authentication, authorization, request validation, and structured API development practices were incorporated to ensure reliable system performance.

The major objectives of my contributions include:

• Development of product management functionalities

• Implementation of RESTful Product APIs

• Support for order processing workflows

• Virtual Try-On workflow development

• Product data management and retrieval

• Backend integration and testing

• Preparation for future AI-driven virtual fashion features

Through these contributions, I gained practical experience in backend application development, API design, database interaction, workflow implementation, and software integration. The work performed helped strengthen the overall backend infrastructure of the Raritone platform while supporting future scalability and feature expansion.

2. SYSTEM OVERVIEW

The Raritone Backend System is a server-side application developed to support the functionality of a virtual fashion platform. It manages the flow of data between users, frontend interfaces, databases, and cloud services while ensuring secure and efficient processing of application requests. The backend is responsible for executing business logic, storing application data, handling API requests, and maintaining system reliability.

Built using Node.js, Express.js, and MongoDB, the backend follows a modular architecture that separates different functionalities into independent components. This design approach improves maintainability, simplifies development, and enables future expansion of platform features.

The system processes requests through a structured workflow where incoming data is validated, authenticated, processed through business logic, and stored or retrieved from the database before a response is returned to the client application. This workflow helps maintain data consistency, application security, and performance.

Core Functional Areas

Product Management

Product management is one of the primary functionalities of the platform. The backend provides APIs that allow administrators to create, update, retrieve, and manage product information efficiently. Product data is organized to support catalog management, filtering operations, and future recommendation systems.

Key Functions:

- Product creation
- Product updates
- Product retrieval
- Product deletion
- Product catalog management

Order Processing

The backend supports order-related operations that enable users to complete purchases and maintain order history. Order management workflows help track purchase information and support transaction processing within the platform.

Key Functions:

- Order creation
- Order retrieval
- Order updates
- Order management
- Purchase workflow support

Virtual Try-On Support

The platform includes backend support for virtual try-on functionality. APIs and workflows have been designed to handle try-on requests and prepare the system for future AI-powered visualization technologies. This functionality aims to improve customer confidence by allowing users to preview fashion products digitally.

Key Functions:

- Try-on request handling
- Product selection workflows
- User image processing support
- Future AI integration support

User Management

The backend provides authentication and authorization mechanisms that enable secure user access. Users can register, log in, manage profiles, and access platform resources according to their permissions.

Key Functions:

- User registration
- User authentication
- Profile management
- Access control

Media Handling

The system supports image upload and storage operations through cloud-based services. Media files are optimized before storage to improve performance and reduce bandwidth usage.

Key Functions:

- Image uploads
- Media optimization
- Cloud storage integration
- Content delivery support

Security Features

Several security mechanisms have been implemented throughout the backend to protect application resources and user information.

Security Measures:

- JWT Authentication
- Role-Based Access Control
- Request Validation
- Rate Limiting
- Error Handling Middleware
- Secure API Access

Scalability and Future Enhancements

The backend architecture has been developed with scalability in mind. The modular design allows future integration of advanced technologies such as artificial intelligence services, caching systems, containerization platforms, and distributed architectures.

Future Enhancements:

- AI-powered recommendations
- Virtual Try-On AI processing
- Redis caching
- Docker deployment
- Kubernetes orchestration
- Microservices architecture

The backend follows RESTful API standards and uses JSON as the primary data exchange format. This approach enables seamless communication between different platform components while supporting future integrations and feature expansion.

3. TECHNOLOGY STACK

The development of the Raritone Backend System required a technology stack capable of supporting secure API development, efficient database management, scalable application architecture, and future AI-driven enhancements. The selected technologies were chosen based on performance, flexibility, ease of development, and compatibility with modern web application requirements.

The backend was primarily built using JavaScript-based technologies, allowing seamless integration between different system components while maintaining a consistent development environment.

Node.js

Node.js was used as the runtime environment for executing backend JavaScript code. Its event-driven architecture and non-blocking I/O operations make it suitable for handling multiple user requests efficiently.

Key Benefits

- Fast execution using Google's V8 Engine
- Asynchronous request processing
- Efficient handling of concurrent users
- Large open-source ecosystem
- Suitable for scalable backend applications

Usage in the Project

Node.js served as the foundation of the backend system and was responsible for running APIs, processing requests, interacting with the database, and managing server-side operations.

Express.js

Express.js was used as the primary backend framework for building RESTful APIs and managing application routes.

Key Benefits

- Simplified route management
- Middleware support

- Faster API development
- Clean project structure
- Easy integration with third-party packages

Usage in the Project

Express.js was utilized to create API endpoints for product management, order processing, authentication workflows, and virtual try-on functionality.

MongoDB

MongoDB was selected as the primary database because of its flexibility and document-oriented architecture.

Key Benefits

- NoSQL document-based storage
- Flexible schema structure
- High scalability
- Efficient data retrieval
- Easy integration with JavaScript applications

Usage in the Project

MongoDB stores information related to products, users, orders, avatars, measurements, wardrobes, wishlists, and virtual try-on data.

Mongoose

Mongoose was used as the Object Data Modeling (ODM) library for MongoDB.

Key Benefits

- Schema-based data modeling
- Data validation support
- Simplified database queries
- Relationship management
- Middleware functionality

Usage in the Project

Mongoose was used to create database schemas, validate incoming data, and manage relationships between collections.

JSON Web Tokens (JWT)

JWT was implemented to provide secure authentication and authorization mechanisms.

Key Benefits

- Stateless authentication
- Secure access control
- Reduced server-side session management
- Token-based verification
- Support for protected routes

Usage in the Project

JWT was used to authenticate users and restrict access to protected APIs based on authorization requirements.

Google OAuth

Google OAuth was integrated to simplify the user authentication process.

Key Benefits

- Faster user registration
- Secure login process
- Reduced password dependency
- Trusted authentication provider

Usage in the Project

Users can log in using their Google accounts without creating separate credentials for the platform.

Cloudinary

Cloudinary was used for cloud-based image storage and media management.

Key Benefits

- Secure image hosting
- Automatic image optimization
- CDN support
- Reduced server storage requirements
- Fast content delivery

Usage in the Project

Cloudinary stores uploaded images used in profiles, avatars, products, and future virtual try-on features.

Multer

Multer was used for handling file uploads within the application.

Key Benefits

- Efficient file processing
- Multipart form-data handling
- Easy integration with Cloudinary
- Improved media upload workflow

Usage in the Project

Multer processes user-uploaded images before they are optimized and stored in cloud storage.

Helmet

Helmet was implemented to improve application security.

Key Benefits

- Protection against common web attacks
- Secure HTTP headers
- Improved application security

- Reduced vulnerability exposure

Usage in the Project

Helmet was used to strengthen the security layer of backend APIs and protect application resources.

Express Rate Limit

Rate limiting was implemented to control excessive API requests.

Key Benefits

- Protection against brute-force attacks
- Prevention of API misuse
- Better server stability
- Improved resource management

Usage in the Project

Rate limiting helps prevent abuse of authentication endpoints and sensitive backend services.

Joi Validation

Joi was used to validate request data before processing.

Key Benefits

- Input validation
- Reduced runtime errors
- Improved data quality
- Consistent API requests

Usage in the Project

Joi validates incoming request payloads to ensure only properly structured data enters the system.

Morgan

Morgan was used for request logging and monitoring during development.

Key Benefits

- Request tracking
- Easier debugging
- Performance monitoring
- Improved maintenance

Usage in the Project

Morgan records API activity and assists developers in troubleshooting application issues.

Technology Stack Summary

The selected technology stack provided a reliable foundation for developing a secure, scalable, and maintainable backend system. Technologies such as Node.js, Express.js, MongoDB, and Mongoose enabled efficient API development, while JWT, Helmet, and Rate Limiting strengthened security. Cloudinary and Multer supported media management requirements, and the overall architecture remains flexible enough to support future enhancements including AI-powered recommendations, virtual try-on processing, and advanced personalization features.

4. SYSTEM ARCHITECTURE

The Raritone Backend System was designed using a modular and layered architecture to ensure efficient request processing, maintainable code organization, and scalability for future enhancements. The architecture separates application responsibilities into different layers, allowing each component to perform a specific function while maintaining loose coupling between modules.

This design approach simplifies development, testing, debugging, and future feature integration. It also provides flexibility for expanding the platform with AI-powered recommendation systems, virtual try-on processing services, and cloud-based deployment solutions.

The backend acts as the central processing unit of the platform, receiving requests from client applications, validating data, executing business logic, interacting with the database, and returning structured responses.

High-Level Architecture

Client Application (Web / Mobile)

↓

API Routes

↓

Middleware Processing

↓

Controllers

↓

Business Logic

↓

Database Models

↓

MongoDB Database

↓

Cloudinary Storage (Media)

↓

Response Returned to Client

Architectural Layers

Client Layer

The client layer consists of web and mobile applications that interact with the backend through REST APIs. Users perform actions such as browsing products, placing orders, managing profiles, and submitting virtual try-on requests.

Responsibilities

- User authentication
- Product browsing
- Product selection
- Cart management
- Order placement
- Profile management
- Virtual try-on interactions

API Routing Layer

The routing layer serves as the entry point for all backend requests. Each API endpoint is mapped to a specific controller responsible for handling the requested operation.

Responsibilities

- Route management
- Request forwarding
- API organization
- Endpoint accessibility

Example Routes

- /api/products
- /api/orders
- /api/auth
- /api/profile
- /api/tryOnRoutes

Middleware Layer

Middleware components process requests before they reach the application logic. This layer helps maintain security, validation, and request consistency throughout the system.

Responsibilities

- User authentication
- Authorization checks
- Request validation
- Error handling
- File upload processing
- Security enforcement

Benefits

- Centralized request processing
- Improved security
- Reduced code duplication
- Consistent validation mechanisms

Controller Layer

Controllers contain the core backend logic responsible for handling application operations. They receive validated requests, execute business processes, communicate with database models, and generate responses.

Major Controller Responsibilities

- Product management operations
- Order processing
- User management
- Wishlist handling
- Avatar management
- Virtual try-on request processing

Contribution Focus

My primary work involved implementing and supporting controller functionality related to:

- Product Module
- Product APIs
- Order Module
- Virtual Try-On Workflows

Business Logic Layer

The business logic layer contains the operational rules that define how data should be processed within the system.

Examples include:

- Product creation workflows
- Product update operations
- Order creation processes
- Product validation rules
- Virtual try-on request handling

Separating business logic from routing improves maintainability and makes future enhancements easier to implement.

Data Model Layer

The data model layer defines the structure of application data using Mongoose schemas. Each schema represents a specific entity and manages validation, relationships, and database interactions.

Major Models

- User Model
- Product Model
- Order Model
- Wishlist Model
- Profile Model
- Avatar Model
- Measurement Model
- Try-On Model

Responsibilities

- Schema definition
- Data validation
- Relationship management
- Database communication

Database Layer

MongoDB serves as the primary database management system for storing and managing application information.

Stored Data

- User accounts
- Product information
- Orders
- Wishlists
- Measurements
- Avatars
- Try-on requests

Benefits

- Flexible document structure
- Fast data retrieval
- High scalability
- Efficient storage management

Media Storage Layer

Cloudinary is integrated into the architecture to manage media assets used throughout the platform.

Responsibilities

- Product image storage
- Avatar image storage
- User-uploaded image management
- Media optimization
- CDN-based delivery

Benefits

- Reduced server storage requirements
- Faster image loading
- Global content delivery
- Improved application performance

Product Management Workflow

One of the major backend functionalities I contributed to was product management.

Workflow:

Administrator Creates Product

↓

Product API Receives Request

↓

Validation Middleware

↓

Product Controller

↓

Product Model

↓

MongoDB Storage

↓

Success Response

This workflow ensures secure and efficient product catalog management throughout the platform.

Virtual Try-On Workflow

The backend architecture also supports future AI-powered virtual try-on functionality.

Workflow:

User Selects Product

↓

Uploads Image

↓

Backend Receives Request

↓

Validation & Processing

↓

Try-On Controller

↓

Database Storage

↓

Future AI Processing Layer

↓

Generated Result Returned

The architecture was designed to allow future integration of image processing and AI visualization services without major modifications to existing modules.

Architectural Benefits

The implemented architecture provides several advantages:

- Modular system design
- Easier maintenance
- Faster feature development
- Improved scalability
- Enhanced security
- Better code organization
- Future AI integration readiness
- Simplified testing and debugging

The layered architecture ensures that the Raritone Backend System remains reliable, maintainable, and adaptable to future business and technological requirements while supporting core functionalities such as product management, order processing, and virtual try-on workflows.

5. PROJECT STRUCTURE

The Raritone Backend System follows a modular folder structure that separates different application responsibilities into dedicated directories. This organization improves readability, simplifies maintenance, and supports collaborative development by allowing team members to work on independent modules without affecting other parts of the system.

The project structure was designed according to common Node.js and Express.js development practices, ensuring scalability and ease of future enhancements.

Overview of Project Organization

The backend is divided into multiple layers, including configuration files, controllers, models, routes, middleware, validators, utility functions, and media processing components. Each layer performs a specific responsibility within the application architecture.

Configuration Layer

The configuration section contains files responsible for establishing connections with external services and application resources.

Main Responsibilities

- Database connection setup
- Cloud service configuration
- Environment initialization
- Application-level settings

Components

Database Configuration

- MongoDB connection management
- Database initialization

Cloudinary Configuration

- Media storage integration
- Cloud service authentication

Benefits

- Centralized configuration management
- Easier maintenance
- Improved deployment flexibility

Controller Layer

Controllers contain the core application logic and act as the processing center for API requests. Each controller handles a specific module and communicates with database models to perform required operations.

Responsibilities

- Process incoming requests
- Execute business logic
- Interact with database models
- Generate API responses

Major Controllers

- Authentication Controller
- Product Controller
- Order Controller
- Profile Controller
- Wishlist Controller
- Avatar Controller
- Virtual Try-On Controller

Contribution Focus

During development, my primary work involved:

- Product Controller
- Order Controller
- Virtual Try-On Controller

These controllers handled product management operations, order processing workflows, and virtual try-on request management.

Model Layer

Models define the structure of data stored within MongoDB and represent different entities used throughout the application.

Responsibilities

- Schema definition
- Data validation
- Relationship management
- Database interaction

Major Models

- User Model

- Product Model
- Order Model
- Profile Model
- Wishlist Model
- Avatar Model
- Measurement Model
- Try-On Model

Contribution Focus

I worked primarily with:

- Product Model
- Order Model
- Try-On Model

These models supported product storage, purchase workflows, and virtual try-on request management.

Routing Layer

The routing layer connects API endpoints to their corresponding controllers. It serves as the entry point for all client requests.

Responsibilities

- API endpoint management
- Request routing
- Controller mapping
- Route organization

Example API Flow

Client Request

↓

Product Route

↓

Product Controller

↓

Product Model

↓

MongoDB Database

↓

Response Returned

Benefits

- Clear API organization
- Easier endpoint maintenance
- Improved scalability

Middleware Layer

Middleware functions provide reusable processing logic that executes before requests reach controllers.

Responsibilities

- Authentication
- Authorization
- Validation
- Error handling
- File upload processing

Advantages

- Reduced code duplication
- Consistent request processing
- Improved security
- Better maintainability

Validation Layer

The validation layer ensures that all incoming request data follows predefined rules before being processed by the application.

Responsibilities

- Request validation
- Data integrity checks
- Error prevention
- Input verification

Examples

- Authentication validation
- Product validation
- Try-On validation
- User registration validation

Benefits

- Improved data quality
- Reduced application errors
- Consistent API behavior

Utility Layer

Utility functions contain reusable helper logic used throughout the backend.

Responsibilities

- Token generation
- Media handling
- Shared helper functions
- Common processing tasks

Benefits

- Code reusability
- Reduced duplication
- Easier maintenance

File Upload and Media Processing

The backend includes a dedicated file upload workflow to support images used in products, avatars, and future virtual try-on features.

Upload Workflow

User Uploads Image

↓

Upload Middleware

↓

Temporary Storage

↓

Cloudinary Processing

↓

Cloud Storage

↓

Response Returned

Benefits

- Efficient media management
- Reduced server storage usage
- Optimized image delivery
- Improved application performance

Project Structure Summary

The modular project structure enables the Raritone Backend System to remain organized, scalable, and easy to maintain. By separating responsibilities into controllers, models, routes, middleware, validators, and utility layers, the system supports efficient development and future expansion. The structure also allowed focused development of Product Management, Order Processing, and Virtual Try-On functionalities while maintaining clean integration with other platform modules.

6. DATABASE DESIGN

The Raritone Backend System uses MongoDB as its primary database due to its flexible document-based structure and scalability. The database is designed to support various platform modules such as user management, product management, order processing, personalization, and virtual try-on functionality.

The system utilizes Mongoose ODM to define schemas, manage relationships, and perform database operations efficiently.

Database Collections

User Collection

Stores account and authentication-related information.

Key Fields

- Name
- Email
- Password
- Role

Purpose

- User authentication
- Authorization management
- Account identification

Product Collection

Stores information related to products available on the platform.

Key Fields

- Product Name
- Category
- Brand
- Price
- Stock

Purpose

- Product catalog management

- Product search and retrieval
- Inventory tracking

Order Collection

Stores purchase and transaction information.

Key Fields

- User ID
- Products
- Total Amount
- Order Status

Purpose

- Order management
- Purchase history tracking
- Transaction processing

Profile Collection

Stores user profile information and preferences.

Purpose

- Personalization
- User preference management

Wishlist Collection

Stores products saved by users for future purchases.

Purpose

- Product bookmarking
- Shopping assistance

Wardrobe Collection

Stores user wardrobe and fashion-related preferences.

Purpose

- Fashion organization
- Future recommendation support

Measurement Collection

Stores body measurement information used for personalization.

Purpose

- Size recommendations
- Better fitting suggestions

Avatar Collection

Stores avatar-related information used for virtual fashion experiences.

Purpose

- Avatar creation
- Virtual try-on support

Try-On Collection

Stores virtual try-on requests and generated results.

Purpose

- Virtual fitting workflows
- Future AI integration

Database Relationships

The database maintains relationships between multiple collections to ensure efficient data management.

Major Relationships

- User → Orders
- User → Wishlist
- User → Profile
- User → Measurements
- User → Avatar
- User → Wardrobe

These relationships help maintain data consistency while supporting personalized user experiences.

Summary

The database design provides a structured and scalable foundation for managing user information, products, orders, and virtual try-on data. Its flexible architecture supports current platform requirements while allowing future integration of recommendation systems and AI-powered fashion services.

7. DATABASE RELATIONSHIPS

The Raritone Backend System uses MongoDB ObjectId references and Mongoose Populate to establish relationships between collections. This approach minimizes data duplication, improves consistency, and enables efficient retrieval of related information across modules.

Major Relationships

User → Wishlist

Relationship Type: One-to-Many

A user can save multiple products to a wishlist for future purchases.

Purpose:

- Product bookmarking
- Personalized shopping experience

User → Orders

Relationship Type: One-to-Many

A user can place multiple orders throughout their usage of the platform.

Purpose:

- Order history management
- Purchase tracking

User → Measurements

Relationship Type: One-to-One

Each user maintains a measurement profile used for personalization.

Purpose:

- Size recommendations
- Better fitting support

User → Avatar

Relationship Type: One-to-Many

Users can manage avatar information for virtual fashion experiences.

Purpose:

- Avatar customization
- Virtual try-on support

Cart → Product

Relationship Type: Many-to-One

Products added to the cart are referenced from the Product collection.

Purpose:

- Product retrieval
- Purchase preparation

Order → Product

Relationship Type: Many-to-One

Orders maintain references to purchased products.

Purpose:

- Order tracking
- Purchase history

Product Mapping → Product

Relationship Type: Many-to-One

Product mappings establish links between related products.

Purpose:

- Recommendation support
- Product compatibility analysis

Mongoose Populate

The project uses Mongoose Populate to retrieve referenced documents automatically. This allows related data such as product details and order information to be fetched efficiently without writing complex queries.

Advantages

- Simplified data retrieval
- Cleaner API responses
- Reduced query complexity
- Better application performance

Summary

The database relationships are designed to maintain data consistency while supporting efficient communication between collections. Using references and Mongoose Populate helps improve scalability, reduce redundancy, and simplify backend development.

8. MODULE DOCUMENTATION

The Raritone Backend System is divided into multiple modules, each responsible for a specific functionality. This modular approach improves maintainability, scalability, and code organization.

Authentication Module

Handles user authentication and authorization.

Key Features

- User Registration
- User Login
- JWT Authentication
- Google OAuth
- Role-Based Access Control

Product Module

The Product Module was one of the primary areas of my contribution. It manages product-related operations within the platform.

Key Features

- Product Creation
- Product Updates
- Product Retrieval
- Product Deletion
- Product Search and Filtering

Benefits

- Efficient catalog management
- Organized product information
- Simplified inventory handling

Profile Module

Manages user profile information and personalization settings.

Key Features

- Profile Management
- Preference Updates
- User Information Storage

Wishlist Module

Allows users to save products for future purchases.

Key Features

- Add Products
- Remove Products
- View Wishlist

Wardrobe Module

Maintains user wardrobe collections and fashion preferences.

Key Features

- Wardrobe Management
- Clothing Organization
- Preference Storage

Cart Module

Provides shopping cart functionality before checkout.

Key Features

- Add to Cart
- Remove from Cart

- View Cart

Order Module

The Order Module supports purchase processing and order management workflows.

Key Features

- Order Creation
- Order Tracking
- Order History
- Order Status Management

Benefits

- Purchase monitoring
- Transaction management
- Improved shopping workflow

Measurement Module

Stores body measurements used for personalization and fitting recommendations.

Key Features

- Save Measurements
- Update Measurements
- Retrieve Measurement Data

Avatar Module

Manages avatar-related information for virtual fashion experiences.

Key Features

- Avatar Storage
- Avatar Management
- Avatar Customization Support

Virtual Try-On Module

The Virtual Try-On Module was another major area of my contribution. It provides backend support for virtual fitting workflows and future AI integration.

Key Features

- Try-On Request Handling
- User Image Processing Support
- Product Selection Workflows
- AI Integration Readiness

Benefits

- Improved product visualization
- Enhanced customer confidence
- Future AI compatibility

Product Mapping Module

Supports relationships between products and recommendation systems.

Key Features

- Product Mapping
- Compatibility Analysis
- Recommendation Support

Summary

The modular architecture allows each component to function independently while maintaining smooth interaction with other modules. My primary contributions focused on the Product Module, Order Module, and Virtual Try-On Module, which form key components of the platform's shopping and personalization experience.

9. API DOCUMENTATION

The Raritone Backend System exposes RESTful APIs that enable communication between client applications and backend services. All APIs use JSON for request and response handling and follow standard REST principles.

Authentication APIs

These APIs manage user authentication and account access.

Method	Endpoint	Purpose
POST	/api/auth/signup	Register a new user
POST	/api/auth/login	User login
POST	/api/auth/logout	User logout
POST	/api/auth/forgot-password	Password recovery
POST	/api/auth/reset-password/:token	Reset password
POST	/api/auth/google	Google OAuth login

Product APIs

The Product Module was one of my primary areas of contribution.

Method	Endpoint	Purpose
POST	/api/products	Create product
GET	/api/products	Retrieve all products
GET	/api/products/:id	Retrieve product by ID
PUT	/api/products/:id	Update product
DELETE	/api/products/:id	Delete product

Features

- Product catalog management
- Product updates
- Product retrieval
- Inventory maintenance

Order APIs

The Order Module manages purchase workflows and order tracking.

Method	Endpoint	Purpose
POST	/api/orders	Create order
GET	/api/orders	Retrieve orders
GET	/api/orders/:id	Retrieve order details
PUT	/api/orders/:id	Update order
DELETE	/api/orders/:id	Delete order

Features

- Order processing
- Order history management
- Transaction tracking

Profile APIs

Method	Endpoint	Purpose
GET	/api/profile	Retrieve profile
PUT	/api/profile/update	Update profile
POST	/api/profile/avatar	Upload avatar

Wishlist APIs

Method	Endpoint	Purpose
POST	/api/wishlist/add	Add product to wishlist
GET	/api/wishlist	Retrieve wishlist
DELETE	/api/wishlist/remove/:productId	Remove wishlist item

Cart APIs

Method	Endpoint	Purpose
POST	/api/cart	Add product to cart
GET	/api/cart	Retrieve cart
DELETE	/api/cart/:productId	Remove cart item

Virtual Try-On APIs

The Virtual Try-On Module supports future AI-based fashion visualization features.

Method	Endpoint	Purpose
POST	/api/tryOnRoutes	Create try-on request
GET	/api/tryOnRoutes	Retrieve try-on results

Features

- Try-on request processing
- User image support
- Future AI integration readiness

API Security

The APIs are protected using multiple security mechanisms:

- JWT Authentication
- Role-Based Authorization
- Request Validation
- Error Handling Middleware
- Rate Limiting

Summary

The API layer provides secure and standardized communication between frontend applications and backend services. My primary contributions involved Product APIs, Order APIs, and Virtual

Try-On APIs, which support product management, purchase workflows, and future virtual fashion experiences.

10. MEDIA PROCESSING WORKFLOW

The Raritone Backend System includes a media processing workflow for handling user-uploaded images efficiently. The workflow ensures that images are validated, optimized, and stored securely before being delivered to users.

Media Processing Flow

Image Upload

↓

Validation

↓

Image Optimization

↓

Cloud Storage

↓

Response Returned

File Upload Handling

The system uses Multer middleware to process image uploads received from users.

Responsibilities

- Receive uploaded files
- Validate file types
- Manage temporary storage

Benefits

- Reliable upload handling
- Simplified file management

- Secure processing workflow

Image Optimization

Uploaded images are optimized before storage to improve performance and reduce file size.

Operations

- Image compression
- Image resizing
- Format optimization

Benefits

- Faster loading times
- Reduced storage usage
- Improved user experience

Cloud Storage

Cloudinary is used as the cloud-based storage solution for images and media assets.

Responsibilities

- Image storage
- Media management
- Content delivery

Benefits

- Secure cloud storage
- High availability
- Scalable media management

Summary

The media processing workflow helps maintain efficient image handling throughout the platform. By combining upload validation, image optimization, and cloud storage, the system ensures reliable media management while supporting future scalability requirements.

11. TESTING DOCUMENTATION

Testing was performed throughout the development process to ensure the backend system functioned correctly, securely, and reliably. Various modules, APIs, database operations, and security mechanisms were verified using Postman and MongoDB validation tools.

Authentication Testing

Authentication workflows were tested to verify secure user access and token generation.

Tested Features

- User Registration
- User Login
- User Logout
- JWT Authentication
- Protected Routes
- Google OAuth Integration

Outcome: Authentication functions operated correctly and secure access was successfully enforced.

API and CRUD Testing

All major Create, Read, Update, and Delete (CRUD) operations were tested across backend modules.

Tested Modules

- Products
- Orders
- Profiles
- Wishlist
- Cart
- Measurements
- Avatars
- Virtual Try-On

Outcome: All API endpoints responded correctly and performed the expected database operations.

Validation Testing

Validation testing ensured that incorrect or incomplete requests were handled properly.

Tested Areas

- Required field validation
- Invalid input handling
- Missing data checks
- Request format verification

Outcome: Invalid requests were rejected with appropriate error responses.

Security Testing

Security mechanisms were verified to ensure proper access control and protection of resources.

Tested Features

- JWT Token Verification
- Protected Endpoints
- Role-Based Access Control (RBAC)
- Authentication Middleware

Outcome: Unauthorized access attempts were successfully blocked.

Database Testing

Database operations were tested to verify data consistency and relationship management.

Tested Areas

- Collection relationships
- ObjectId references
- Mongoose Populate operations
- Data retrieval and storage

Outcome: Database interactions were completed successfully without data inconsistencies.

Integration Testing

Integration testing verified communication between different backend layers.

Tested Areas

- Routes and Controllers
- Controllers and Models
- Database Connectivity
- API Response Generation

Outcome: All components interacted correctly and produced expected results.

Testing Summary

The backend system was successfully tested across authentication, API operations, validation, security, database management, and module integration. The results demonstrated stable performance, reliable functionality, and secure communication between system components. No major issues were identified during testing, and all core modules functioned as expected.

12. SCALABILITY RESEARCH

As the Raritone platform grows, the backend infrastructure should be capable of supporting increased user traffic, larger product catalogs, higher transaction volumes, and future AI-powered services. Several scalability technologies and approaches were researched for potential future implementation.

Redis Caching

Redis can be used to store frequently accessed data in memory, reducing database queries and improving response times.

Potential Applications

- Product data caching
- User session management
- Frequently accessed API responses

Benefits

- Faster performance
- Reduced database load

- Improved user experience

Message Queue Systems

Queue-based architectures can help process time-consuming tasks asynchronously.

Potential Applications

- Virtual Try-On processing
- Avatar generation
- Email notifications
- Image processing

Technologies Explored

- BullMQ
- RabbitMQ
- Apache Kafka

Benefits

- Better resource utilization
- Improved scalability
- Faster request handling

Containerization and Deployment

Container technologies provide consistent deployment across development and production environments.

Technologies Explored

- Docker
- Kubernetes

Benefits

- Easier deployment
- Automatic scaling
- High availability
- Improved infrastructure management

Database Scaling

MongoDB Atlas offers cloud-based database management with built-in scalability features.

Features

- Automatic scaling
- Backup management
- Monitoring tools
- Multi-region deployment

Benefits

- Improved reliability
- High availability
- Reduced maintenance effort

Summary

The scalability research highlighted several technologies that can support future platform growth. Solutions such as Redis, message queues, Docker, Kubernetes, and MongoDB Atlas can help improve performance, reliability, and scalability while preparing the system for advanced features such as AI-powered recommendations and virtual try-on processing.

13. DEPLOYMENT DOCUMENTATION

The deployment strategy for the Raritone Backend was researched to identify suitable cloud platforms for hosting, database management, and media storage. The objective was to evaluate deployment solutions that support scalability, reliability, and ease of maintenance.

Deployment Platforms Studied

Render

Render is a cloud platform that supports deployment of Node.js applications with minimal configuration.

Advantages

- Easy deployment process
- Automatic SSL support

- Continuous deployment integration
- Developer-friendly interface

Use Case

- Small and medium-scale applications

Railway

Railway provides a simplified deployment environment for rapid development and testing.

Advantages

- Quick setup
- Simple configuration
- Easy project deployment

Use Case

- Development and staging environments

AWS (Amazon Web Services)

AWS offers enterprise-level cloud infrastructure and scalability.

Services Explored

- EC2
- S3
- CloudFront
- Elastic Load Balancer
- EKS

Advantages

- High scalability
- Global infrastructure
- Reliable cloud services

Use Case

- Large-scale production deployments

Supporting Cloud Services

MongoDB Atlas

- Managed cloud database
- Automated backups
- Monitoring and scaling support

Cloudinary

- Cloud-based image storage
- Media optimization
- CDN delivery

Summary

The deployment research identified Render and Railway as suitable options for development and testing, while AWS provides a scalable solution for future production deployment. MongoDB Atlas and Cloudinary complement the infrastructure by providing database and media management services.

14. AI INTEGRATION PLANNING

The backend architecture was designed with future AI integration in mind. The modular structure allows intelligent features to be added without major modifications to the existing system.

AI Avatar Generation

Future AI services can be integrated to generate personalized avatars based on user-uploaded images and profile information.

Expected Benefits

- Personalized avatars
- Enhanced user experience
- Improved platform engagement

AI-Powered Recommendations

User preferences, measurements, wardrobe data, and shopping activity can be utilized to generate personalized product recommendations.

Expected Benefits

- Better product discovery
- Personalized shopping experience
- Increased user engagement

Virtual Try-On AI

One of the key future enhancements is AI-powered virtual try-on functionality. The system architecture already includes support for processing try-on requests and storing generated outputs.

Expected Benefits

- Improved product visualization
- Reduced purchase uncertainty
- Better customer satisfaction

Real-Time Processing

Future implementations may utilize Socket.IO to provide real-time updates for AI-related operations.

Potential Events

- Try-On Processing Updates
- Avatar Generation Status
- Recommendation Updates

Benefits

- Instant notifications
- Improved user interaction
- Faster feedback to users

Summary

The Raritone Backend System has been developed with a future-ready architecture that supports AI-powered avatars, recommendation systems, and virtual try-on technologies. These planned enhancements can further improve personalization, user engagement, and the overall shopping experience.

15. TEAM CONTRIBUTIONS

The Raritone Backend Project was developed through a collaborative team effort, with each member contributing to specific modules and supporting activities such as testing, documentation, integration, and system design.

Individual Contributions

Nandini Bhimineni

Role: Backend Developer & Integration Coordinator

Key Contributions

- Backend integration
- Cart and Order modules
- Measurement and Avatar modules
- Product Mapping module
- Database relationship design
- API testing and documentation support

Meka Hrishi Teja Chowdary

Key Contributions

- Product Module development
- Product API implementation
- Order Module support
- Virtual Try-On workflow development
- Product management operations

Bharath Kumar

Key Contributions

- Wishlist module
- Authentication support
- API validation
- Security enhancements
- Backend testing support

Vishnu Vardhan Reddy

Key Contributions

- Wishlist module
- Virtual Try-On support
- Authentication assistance
- API testing and workflow validation

Vagdevi Malineni

Key Contributions

- Authentication module
- JWT implementation
- Role-Based Access Control (RBAC)
- Profile and Wardrobe modules
- Security feature integration

Nainisha Bandari

Key Contributions

- Profile Module development
- Wardrobe Module development
- User profile management
- User preference management

Collaborative Contributions

In addition to individual responsibilities, the team collectively contributed to:

- REST API development
- MongoDB database design
- JWT Authentication
- Google OAuth integration
- Cloudinary media management
- Input validation and error handling
- Postman API testing
- GitHub collaboration and version control
- Backend architecture design
- Technical documentation
- Scalability and deployment research

Summary

The successful development of the Raritone Backend System was achieved through effective collaboration among team members. Each contributor was responsible for specific modules while also participating in testing, integration, and system improvements, resulting in a scalable and well-structured backend architecture.

16. MY CONTRIBUTIONS

Contributor Information

Name: Meka Hrishi Teja Chowdary

Role: Backend Developer

Overview

Throughout the Raritone Backend Project, I contributed to the design, development, testing, and integration of core backend functionalities related to product management, order processing, and virtual try-on workflows. My responsibilities focused on building scalable APIs, implementing business logic, managing product operations, and supporting innovative virtual fashion features.

These contributions helped establish a reliable backend foundation for product handling, purchase workflows, and future AI-powered virtual try-on capabilities within the platform.

Modules Developed

Product Module

Responsibilities

- Product schema implementation
- Product API development
- Product CRUD operations
- Product management workflows
- Product retrieval and filtering

Contribution

Developed the Product Module to manage the platform's product catalog. Implemented functionalities for creating, updating, retrieving, and deleting products while ensuring efficient product management and smooth interaction between the frontend and backend systems.

Key Features Implemented

- Product creation
- Product updates
- Product deletion
- Product retrieval
- Product management operations
- Product data validation

Product APIs

Responsibilities

- REST API development
- API endpoint implementation
- Request processing
- Response generation
- API testing

Contribution

Designed and implemented Product APIs following RESTful principles. Developed endpoints that allow frontend applications to interact securely and efficiently with product-related backend services.

APIs Implemented

Create Product

- POST /api/products

Get All Products

- GET /api/products

Get Product By ID

- GET /api/products/:id

Update Product

- PUT /api/products/:id

Delete Product

- DELETE /api/products/:id

Benefits

- Efficient product management
- Simplified frontend integration
- Standardized API communication
- Improved maintainability

Order Module

Responsibilities

- Order workflow implementation
- Order API support
- Order management logic
- Database integration
- Purchase processing support

Contribution

Participated in the development of the Order Module to support customer purchase workflows. Contributed to order creation, retrieval, and management processes while ensuring proper interaction between products, users, and order records.

Key Features

- Order creation
- Order retrieval
- Order management
- Purchase tracking support
- Order history functionality

Benefits

- Efficient order processing
- Transaction management
- Improved purchase workflow
- Enhanced user experience

Virtual Try-On Module

Responsibilities

- Virtual try-on workflow support
- Backend process development
- API integration
- Request handling
- Future AI compatibility

Contribution

Contributed to the Virtual Try-On Module by developing backend workflows and supporting APIs required for virtual fitting functionality. The implementation was designed to accommodate future AI-based image processing and visualization systems.

Features Supported

- Try-on request handling
- User image processing support
- Product selection workflows
- AI integration readiness

Benefits

- Improved product visualization
- Enhanced shopping confidence
- Reduced purchase uncertainty
- Future AI scalability

Product Management Operations

Responsibilities

- Product lifecycle management
- Product information maintenance
- Database operations
- Inventory-related support

Contribution

Managed product-related operations to ensure accurate storage, retrieval, and maintenance of product information. Assisted in creating a structured and scalable product management system that supports future recommendation and personalization features.

Achievements

- Organized product catalog structure

- Improved product accessibility
- Supported scalable product operations
- Enhanced system maintainability

Try-On Workflow Development

Responsibilities

- Workflow design support
- Backend integration
- Request processing
- System coordination

Contribution

Participated in developing the backend workflow for virtual try-on functionality. Assisted in designing request processing mechanisms and integration pathways that enable future AI-powered visualization systems.

Workflow

User Selects Product

↓

Uploads Image

↓

Backend Receives Request

↓

Try-On Processing Workflow

↓

Result Generation

↓

Response Returned

Benefits

- Structured try-on processing
- Improved user experience
- Future AI compatibility

- Scalable architecture

Testing Activities

Tools Used

- Postman
- MongoDB Database Tools

Testing Areas

- Product APIs
- Order APIs
- CRUD Operations
- Endpoint Validation
- Response Verification
- Workflow Testing

Contribution

Conducted testing activities to verify functionality, reliability, and performance of implemented modules. Ensured APIs responded correctly and met project requirements.

Integration Activities

Responsibilities

- Module integration
- API integration
- Route configuration
- Backend coordination

Contribution

Collaborated with team members during integration of backend modules to ensure seamless communication between products, orders, authentication systems, and virtual try-on functionalities.

Key Achievements

- Developed Product Module functionalities.
- Implemented Product APIs using RESTful architecture.
- Contributed to Order Module development.
- Supported Virtual Try-On Module implementation.

- Participated in Try-On Workflow development.
- Performed API testing and validation.
- Assisted in backend integration activities.
- Contributed to scalable backend architecture development.

Summary

My contributions primarily focused on Product Management, Product APIs, Order Processing, and Virtual Try-On functionalities. Through backend development, testing, workflow implementation, and system integration activities, I helped establish a scalable and maintainable backend environment for the Raritone platform. The implemented modules provide a strong foundation for future enhancements, including AI-powered virtual try-on experiences and intelligent product management systems.